CROP DISEASE DETECTION APP

Detailed Build Prompt Library

PHASE 4 — Integration, Testing, Deployment & Docs

Prompts 166–220 · 55 Fully Detailed Prompts

Prepared for: Priyanshu Kumar Gupta & Team
(Sadhana Bag, Kumar Gaurav, Rajdeep Kumar)

Part 4 of 4 — execute this phase fully before opening the next

Phase Overview

This phase connects the three finished pieces into one live product, proves it works, hardens it, ships it, and prepares the written and spoken deliverables.

Reference: Full Project File Structure

Every prompt in this document references paths from the following master file tree. Keep it visible while executing.

```
crop-disease-detection/
|-- README.md
|-- ml/
|   |-- requirements.txt
|   |-- dataset/
|   |   |-- raw/
|   |   |-- filtered/
|   |   |-- split/
|   |   |   |-- train/<class>/
|   |   |   |-- val/<class>/
|   |   |   `-- test/<class>/
|   |   `-- real_world_sample/<class>/
|   |-- notebooks/
|   |   |-- 00_colab_setup.ipynb
|   |   |-- 01_dataset_exploration.ipynb
|   |   `-- 02_preprocessing_test.ipynb
|   |-- preprocessing/
|   |   |-- loader.py
|   |   |-- augmentation.py
|   |   |-- dataset_pipeline.py
|   |   `-- class_weights.py
|   |-- training/
|   |   |-- baseline_cnn.py
|   |   |-- train_baseline.py
|   |   |-- transfer_mobilenetv2.py
|   |   |-- train_transfer.py
|   |   `-- transfer_resnet50.py
|   |-- evaluation/
|   |   |-- evaluate.py
|   |   |-- confusion_matrix.py
|   |   |-- compare_models.py
|   |   |-- confidence_threshold_experiment.py
|   |   `-- real_world_eval.py
```

```
|   `-- models/
|       |-- crop_disease_model.keras
|       `-- class_names.json
|
|-- backend/
|   |-- requirements.txt
|   |-- .env.example
|   |-- alembic/
|   |-- scripts/
|   |   `-- reset_db.py
|   |-- tests/
|   |   |-- conftest.py
|   |   |-- test_auth.py
|   |   |-- test_users.py
|   |   |-- test_predict.py
|   |   `-- test_history_feedback.py
|   `-- app/
|       |-- main.py
|       |-- core/
|       |   |-- config.py
|       |   |-- security.py
|       |   `-- dependencies.py
|       |-- db/
|       |   |-- session.py
|       |   `-- seed_diseases.py
|       |-- models/
|       |   |-- user.py
|       |   |-- prediction.py
|       |   |-- disease.py
|       |   `-- feedback.py
|       |-- schemas/
|       |   |-- auth.py
|       |   |-- user.py
|       |   `-- prediction.py
|       |-- routes/
|       |   |-- auth.py
|       |   |-- users.py
|       |   |-- predict.py
|       |   `-- diseases.py
|       |-- services/
|       |   |-- model_service.py
|       |   `-- preprocessing_service.py
|       `-- disease_data/
|
|-- frontend/
|   |-- package.json
|   |-- tailwind.config.js
|   |-- index.html
|   `-- src/
|       |-- App.tsx
|       |-- components/
|       |   |-- ui/
|       |   |   |-- Button.tsx
|       |   |   |-- Card.tsx
|       |   |   |-- Input.tsx
|       |   |   |-- Avatar.tsx
|       |   |   `-- Spinner.tsx
|       |   |-- StatsCard.tsx
|       |   |-- RecentScansTable.tsx
|       |   |-- ProtectedRoute.tsx
|       |   `-- Layout.tsx
|       |-- pages/
|       |   |-- Signup.tsx
|       |   |-- Login.tsx
|       |   |-- Dashboard.tsx
|       |   |-- NewScan.tsx
|       |   |-- Analyzing.tsx
|       |   |-- Result.tsx
```

```
|       |   |-- History.tsx
|       |   |-- Profile.tsx
|       |   `-- About.tsx
|       |-- services/
|       |   `-- api.ts
|       |-- store/
|       |   `-- authStore.ts
|       |-- hooks/
|       `-- assets/
|
`-- docs/
    |-- SETUP.md
    |-- ARCHITECTURE.md
    |-- API_REFERENCE.md
    |-- DATABASE_SCHEMA.md
    |-- SECURITY.md
    |-- PERFORMANCE.md
    |-- TESTING_REPORT.md
    `-- DEMO_SCRIPT.md
```

Detailed Prompts

4.A Frontend-Backend Integration

PROMPT 166 — Wire the Signup page end-to-end against the live backend and verify a new user lands on a wo...

Objective: Wire the Signup page end-to-end against the live backend and verify a new user lands on a working dashboard.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Wire the Signup page end-to-end against the live backend and verify a new user lands on a working dashboard.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 167 — Wire the Login page end-to-end and verify JWT persistence across a page refresh

Objective: Wire the Login page end-to-end and verify JWT persistence across a page refresh.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Wire the Login page end-to-end and verify JWT persistence across a page refresh.
- Test against the real running backend (not a mock) for this verification.

- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 168 — Wire the full scan flow end-to-end

Objective: Wire the full scan flow end-to-end: upload -> analyzing -> result -> saved to history.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Wire the full scan flow end-to-end: upload -> analyzing -> result -> saved to history.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 169 — Verify the dashboard stats and recent-scans update immediately after a new scan

Objective: Verify the dashboard stats and recent-scans update immediately after a new scan.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Verify the dashboard stats and recent-scans update immediately after a new scan.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 170 — Verify the History page correctly paginates and filters against real backend data

Objective: Verify the History page correctly paginates and filters against real backend data.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Verify the History page correctly paginates and filters against real backend data.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 171 — Verify the Profile edit flow persists changes and reflects them immediately in the navbar av...

Objective: Verify the Profile edit flow persists changes and reflects them immediately in the navbar avatar/name.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Verify the Profile edit flow persists changes and reflects them immediately in the navbar avatar/name.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 172 — Verify that logging in as a second test user shows a completely separate dashboard/history (...

Objective: Verify that logging in as a second test user shows a completely separate dashboard/history (no data leakage between accounts).

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Verify that logging in as a second test user shows a completely separate dashboard/history (no data leakage between accounts).
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 173 — Fix any API-contract mismatches discovered during integration (field naming, date formats, e...

Objective: Fix any API-contract mismatches discovered during integration (field naming, date formats, error shapes).

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Fix any API-contract mismatches discovered during integration (field naming, date formats, error shapes).
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 174 — Add global frontend error boundaries and a friendly fallback UI for unexpected failures

Objective: Add global frontend error boundaries and a friendly fallback UI for unexpected failures.

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add global frontend error boundaries and a friendly fallback UI for unexpected failures.
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

PROMPT 175 — Add environment-based API base URL configuration (.env for local vs

Objective: Add environment-based API base URL configuration (`.env` for local vs. production).

Why This Matters: This is where the three independently-built pieces (ML model, backend, frontend) become one actual product. Integration bugs — contract mismatches, auth not wired correctly, stale data — are far cheaper to catch here than during the live demo.

Primary File(s): `.env`

Detailed Requirements

- Add environment-based API base URL configuration (`.env` for local vs. production).
- Test against the real running backend (not a mock) for this verification.
- If a mismatch is found between what the frontend expects and what the backend returns, fix the contract and re-verify — don't just patch around it on one side.
- Use two distinct test user accounts wherever data-isolation is being verified.
- Record the verification steps taken (even briefly) so they can be re-run after future changes.

Technical Specification

Spans `frontend/src/` (the calling side) and `backend/app/routes/` (the serving side) for whichever flow is named in the objective above.

Acceptance Criteria

- ☐ The flow described in the objective works end-to-end against the real backend with no console errors.
- ☐ Data shown in the UI matches what is actually stored in the database for that user.
- ☐ Switching accounts shows correctly isolated, different data.

Edge Cases & Notes

- Check what happens if the backend is temporarily unreachable during this flow — the frontend should show an error state, not hang or crash.

4.B End-to-End Testing

PROMPT 176 — Write an end-to-end test script (Playwright or Cypress) covering

Objective: Write an end-to-end test script (Playwright or Cypress) covering: signup -> login -> scan -> view result -> logout.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just “worked once when I tried it” — and they give concrete evidence for the final report's evaluation section.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Write an end-to-end test script (Playwright or Cypress) covering: signup -> login -> scan -> view result -> logout.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 177 — Write an end-to-end test covering the "uncertain result" path using a deliberately ambiguous...

Objective: Write an end-to-end test covering the "uncertain result" path using a deliberately ambiguous test image.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just “worked once when I tried it” — and they give concrete evidence for the final report's evaluation section.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Write an end-to-end test covering the "uncertain result" path using a deliberately ambiguous test image.

- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 178 — Write an end-to-end test verifying protected routes redirect unauthenticated users to /login

Objective: Write an end-to-end test verifying protected routes redirect unauthenticated users to `/login`.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): `/login`

Detailed Requirements

- Write an end-to-end test verifying protected routes redirect unauthenticated users to `/login`.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 179 — Write an end-to-end test verifying user data isolation between two different accounts

Objective: Write an end-to-end test verifying user data isolation between two different accounts.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Write an end-to-end test verifying user data isolation between two different accounts.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 180 — Run the full backend test suite (pytest) and fix any failing tests

Objective: Run the full backend test suite (`pytest`) and fix any failing tests.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Run the full backend test suite (`pytest`) and fix any failing tests.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.

- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 181 — Perform manual QA on real mobile devices (Android and iOS browsers) for the upload flow

Objective: Perform manual QA on real mobile devices (Android and iOS browsers) for the upload flow.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Perform manual QA on real mobile devices (Android and iOS browsers) for the upload flow.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 182 — Load-test the /predict endpoint with concurrent requests to confirm the model-loading sing...

Objective: Load-test the `/predict` endpoint with concurrent requests to confirm the model-loading singleton performs acceptably.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): `/predict`

Detailed Requirements

- Load-test the `/predict` endpoint with concurrent requests to confirm the model-loading singleton performs acceptably.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.
- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

PROMPT 183 — Document all test results and known issues in docs/TESTING_REPORT.md

Objective: Document all test results and known issues in `docs/TESTING_REPORT.md`.

Why This Matters: Automated and manual tests at this stage are what let the team trust the system is actually correct — not just "worked once when I tried it" — and they give concrete evidence for the final report's evaluation section.

Primary File(s): `docs/TESTING_REPORT.md`

Detailed Requirements

- Document all test results and known issues in `docs/TESTING_REPORT.md`.
- Automated tests should be runnable with a single command and not depend on manual setup steps beyond what's documented.
- Cover both the happy path and at least one realistic failure path for the flow under test.
- Use a dedicated test database/environment, never the production database, for destructive test scenarios.

- Record results (pass/fail, and for manual QA, device/browser used) in the testing report.

Technical Specification

Backend tests use `pytest` against `backend/tests/`. Frontend/E2E tests use Playwright or Cypress against the full running stack (frontend + backend + database).

Acceptance Criteria

- ☐ The test(s) described in the objective exist, run, and pass against the current codebase.
- ☐ A deliberately-broken version of the feature causes the test to fail (confirming the test actually tests something).
- ☐ Results are recorded in `docs/TESTING_REPORT.md`.

Edge Cases & Notes

- Note any flaky behaviour (a test that sometimes passes, sometimes fails with no code change) rather than ignoring it.

4.C Security Hardening

PROMPT 184 — Audit all protected routes to confirm none rely on client-supplied user IDs instead of the J...

Objective: Audit all protected routes to confirm none rely on client-supplied user IDs instead of the JWT-derived identity.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Audit all protected routes to confirm none rely on client-supplied user IDs instead of the JWT-derived identity.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.
- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

PROMPT 185 — Confirm password hashes are never included in any API response, log line, or error message

Objective: Confirm password hashes are never included in any API response, log line, or error message.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Confirm password hashes are never included in any API response, log line, or error message.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.

- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

PROMPT 186 — Add HTTPS-only cookie flags / secure token storage configuration for production

Objective: Add HTTPS-only cookie flags / secure token storage configuration for production.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add HTTPS-only cookie flags / secure token storage configuration for production.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.
- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

PROMPT 187 — Add basic rate-limiting to /auth/login and /predict to reduce abuse risk

Objective: Add basic rate-limiting to `/auth/login` and `/predict` to reduce abuse risk.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): `/auth/login`, `/predict`

Detailed Requirements

- Add basic rate-limiting to `/auth/login` and `/predict` to reduce abuse risk.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.
- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

PROMPT 188 — Review file-upload handling for path-traversal and malicious-file risks

Objective: Review file-upload handling for path-traversal and malicious-file risks.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Review file-upload handling for path-traversal and malicious-file risks.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.
- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

PROMPT 189 — Document the security measures taken in docs/SECURITY.md

Objective: Document the security measures taken in `docs/SECURITY.md`.

Why This Matters: Real user accounts mean real responsibility. This audit step exists because the project report explicitly commits to never letting one user access another's data and to never storing credentials unsafely — this prompt is where that commitment is verified, not just assumed.

Primary File(s): `docs/SECURITY.md`

Detailed Requirements

- Document the security measures taken in `docs/SECURITY.md`.
- Review actual code (not just documentation) for the specific concern named in the objective.
- Where a gap is found, fix it immediately rather than only noting it for later.
- Confirm the fix with a concrete test (e.g. attempt the exact attack/misuse being guarded against and confirm it is blocked).
- Record the finding and fix in `docs/SECURITY.md`.

Technical Specification

Applies to `backend/app/core/security.py`, `backend/app/core/dependencies.py`, and every route in `backend/app/routes/` relevant to the objective above.

Acceptance Criteria

- ☐ The specific security property described in the objective is verified true by an explicit, repeatable check — not assumed.
- ☐ Any gap found during this audit is fixed and re-verified before moving on.
- ☐ `docs/SECURITY.md` is updated to reflect this check.

Edge Cases & Notes

- If a fix here changes an API contract (e.g. adding a required header), update the frontend and documentation in the same pass so they don't drift out of sync.

4.D Performance Pass

PROMPT 190 — Profile the /predict endpoint latency and optimize preprocessing/inference if it exceeds a...

Objective: Profile the `/predict` endpoint latency and optimize preprocessing/inference if it exceeds an acceptable response-time budget.

Why This Matters: A personalized app that feels slow undermines the entire product goal of feeling like a user's own responsive assistant. This prompt exists to put real numbers against that feeling and fix the worst offenders before deployment.

Primary File(s): `/predict`

Detailed Requirements

- Profile the `/predict` endpoint latency and optimize preprocessing/inference if it exceeds an acceptable response-time budget.
- Measure before optimizing — record a concrete baseline number (ms, KB, etc.) before making any change.
- Re-measure after the change and record the delta.
- Prefer the simplest fix that meaningfully improves the number over a complex one with marginal gains.
- Don't trade away correctness (e.g. aggressive caching that serves stale user-specific data) for speed.

Technical Specification

Relevant surfaces: `backend/app/services/model_service.py` + `/predict` route latency; `frontend/` Vite build output size; client-side image handling before upload.

Acceptance Criteria

- ☐ A documented before/after measurement shows a real improvement for the metric described in the objective.
- ☐ No functional regression is introduced by the optimization (re-run relevant tests).
- ☐ Results are recorded in `docs/PERFORMANCE.md`.

Edge Cases & Notes

- Confirm the optimization still behaves correctly on a slow/throttled network connection, not just on localhost.

PROMPT 191 — Add response caching where appropriate (e

Objective: Add response caching where appropriate (e.g. `/diseases` reference data).

Why This Matters: A personalized app that feels slow undermines the entire product goal of feeling like a user's own responsive assistant. This prompt exists to put real numbers against that feeling and fix the worst offenders before deployment.

Primary File(s): `/diseases`

Detailed Requirements

- Add response caching where appropriate (e.g. `/diseases` reference data).
- Measure before optimizing — record a concrete baseline number (ms, KB, etc.) before making any change.
- Re-measure after the change and record the delta.
- Prefer the simplest fix that meaningfully improves the number over a complex one with marginal gains.

- Don't trade away correctness (e.g. aggressive caching that serves stale user-specific data) for speed.

Technical Specification

Relevant surfaces: `backend/app/services/model_service.py` + `/predict` route latency; `frontend/` Vite build output size; client-side image handling before upload.

Acceptance Criteria

- ☐ A documented before/after measurement shows a real improvement for the metric described in the objective.
- ☐ No functional regression is introduced by the optimization (re-run relevant tests).
- ☐ Results are recorded in `docs/PERFORMANCE.md`.

Edge Cases & Notes

- Confirm the optimization still behaves correctly on a slow/throttled network connection, not just on localhost.

PROMPT 192 — Optimize frontend bundle size (code-splitting routes, lazy-loading the Result/History pages)

Objective: Optimize frontend bundle size (code-splitting routes, lazy-loading the Result/History pages).

Why This Matters: A personalized app that feels slow undermines the entire product goal of feeling like a user's own responsive assistant. This prompt exists to put real numbers against that feeling and fix the worst offenders before deployment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Optimize frontend bundle size (code-splitting routes, lazy-loading the Result/History pages).
- Measure before optimizing — record a concrete baseline number (ms, KB, etc.) before making any change.
- Re-measure after the change and record the delta.
- Prefer the simplest fix that meaningfully improves the number over a complex one with marginal gains.
- Don't trade away correctness (e.g. aggressive caching that serves stale user-specific data) for speed.

Technical Specification

Relevant surfaces: `backend/app/services/model_service.py` + `/predict` route latency; `frontend/` Vite build output size; client-side image handling before upload.

Acceptance Criteria

- ☐ A documented before/after measurement shows a real improvement for the metric described in the objective.
- ☐ No functional regression is introduced by the optimization (re-run relevant tests).
- ☐ Results are recorded in `docs/PERFORMANCE.md`.

Edge Cases & Notes

- Confirm the optimization still behaves correctly on a slow/throttled network connection, not just on localhost.

PROMPT 193 — Add image compression/resizing on the client before upload to reduce payload size

Objective: Add image compression/resizing on the client before upload to reduce payload size.

Why This Matters: A personalized app that feels slow undermines the entire product goal of feeling like a user's own responsive assistant. This prompt exists to put real numbers against that feeling and fix the worst offenders before deployment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Add image compression/resizing on the client before upload to reduce payload size.
- Measure before optimizing — record a concrete baseline number (ms, KB, etc.) before making any change.
- Re-measure after the change and record the delta.
- Prefer the simplest fix that meaningfully improves the number over a complex one with marginal gains.
- Don't trade away correctness (e.g. aggressive caching that serves stale user-specific data) for speed.

Technical Specification

Relevant surfaces: `backend/app/services/model_service.py` + `/predict` route latency; `frontend/` Vite build output size; client-side image handling before upload.

Acceptance Criteria

- ☐ A documented before/after measurement shows a real improvement for the metric described in the objective.
- ☐ No functional regression is introduced by the optimization (re-run relevant tests).
- ☐ Results are recorded in `docs/PERFORMANCE.md`.

Edge Cases & Notes

- Confirm the optimization still behaves correctly on a slow/throttled network connection, not just on localhost.

PROMPT 194 — Re-run and document before/after performance numbers in docs/PERFORMANCE.md

Objective: Re-run and document before/after performance numbers in `docs/PERFORMANCE.md`.

Why This Matters: A personalized app that feels slow undermines the entire product goal of feeling like a user's own responsive assistant. This prompt exists to put real numbers against that feeling and fix the worst offenders before deployment.

Primary File(s): `docs/PERFORMANCE.md`

Detailed Requirements

- Re-run and document before/after performance numbers in `docs/PERFORMANCE.md`.
- Measure before optimizing — record a concrete baseline number (ms, KB, etc.) before making any change.
- Re-measure after the change and record the delta.
- Prefer the simplest fix that meaningfully improves the number over a complex one with marginal gains.
- Don't trade away correctness (e.g. aggressive caching that serves stale user-specific data) for speed.

Technical Specification

Relevant surfaces: `backend/app/services/model_service.py` + `/predict` route latency; `frontend/` Vite build output size; client-side image handling before upload.

Acceptance Criteria

- ☐ A documented before/after measurement shows a real improvement for the metric described in the objective.
- ☐ No functional regression is introduced by the optimization (re-run relevant tests).
- ☐ Results are recorded in `docs/PERFORMANCE.md`.

Edge Cases & Notes

- Confirm the optimization still behaves correctly on a slow/throttled network connection, not just on localhost.

4.E Backend Deployment

PROMPT 195 — Provision a managed PostgreSQL instance (e

Objective: Provision a managed PostgreSQL instance (e.g. Render/Railway/Supabase) for production.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Provision a managed PostgreSQL instance (e.g. Render/Railway/Supabase) for production.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.
- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

PROMPT 196 — Configure production environment variables (DATABASE_URL, JWT_SECRET, MODEL_PATH) on the hos...

Objective: Configure production environment variables (DATABASE_URL, JWT_SECRET, MODEL_PATH) on the hosting platform.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Configure production environment variables (DATABASE_URL, JWT_SECRET, MODEL_PATH) on the hosting platform.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.

- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

PROMPT 197 — Deploy the FastAPI backend (Docker or platform-native build) and confirm GET /health respo...

Objective: Deploy the FastAPI backend (Docker or platform-native build) and confirm `GET /health` responds correctly.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): `GET /health`

Detailed Requirements

- Deploy the FastAPI backend (Docker or platform-native build) and confirm `GET /health` responds correctly.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.
- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

PROMPT 198 — Run production database migrations and seed the diseases table in the production database

Objective: Run production database migrations and seed the `diseases` table in the production database.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Run production database migrations and seed the `diseases` table in the production database.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.
- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

PROMPT 199 — Upload/verify the trained model file is accessible to the deployed backend instance

Objective: Upload/verify the trained model file is accessible to the deployed backend instance.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Upload/verify the trained model file is accessible to the deployed backend instance.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.

- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

PROMPT 200 — Smoke-test all API endpoints against the live production URL

Objective: Smoke-test all API endpoints against the live production URL.

Why This Matters: This step takes the backend from “works on my machine” to a live, reachable service the deployed frontend and real users can actually hit — the final proof the system works outside the development environment.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Smoke-test all API endpoints against the live production URL.
- Never commit real production secrets to the repository; set them via the hosting platform's environment-variable configuration.
- Confirm the production database is a genuinely separate instance from any local/development database.
- Re-run migrations against production explicitly as a deliberate step, not assumed to have happened automatically.
- Verify the deployed service's `/health` endpoint before considering this step done.

Technical Specification

Target platform: a managed Python/FastAPI host (e.g. Render/Railway) plus a managed PostgreSQL instance. Required env vars: DATABASE_URL, JWT_SECRET, MODEL_PATH.

Acceptance Criteria

- ☐ The deployed backend responds correctly to `GET /health` from outside the local network.
- ☐ At least one full authenticated flow (register -> login -> predict) succeeds against the live URL.
- ☐ No secrets are visible in the deployed build's logs or client-exposed config.

Edge Cases & Notes

- Confirm behaviour if the deployed instance restarts (the model should reload correctly at startup, not require a manual trigger).

4.F Frontend Deployment

PROMPT 201 — Configure the frontend's production .env with the live backend API URL

Objective: Configure the frontend's production `.env` with the live backend API URL.

Why This Matters: This step makes the personalized app actually reachable by real users at a real URL, pointed at the real deployed backend — completing the full path described in the system architecture chapter of the report.

Primary File(s): `.env`

Detailed Requirements

- Configure the frontend's production `.env` with the live backend API URL.
- Confirm the production build uses the deployed backend's URL, not `localhost`.
- Verify HTTPS is active end-to-end (both frontend and backend) before considering the deployment complete.
- Confirm environment-specific config is injected at build/deploy time, not hard-coded in source.
- Keep a rollback path available (previous deployment/build) in case the new deploy breaks something.

Technical Specification

Target platform: Vercel (or equivalent static/SSR host for a Vite React app). Required env var: production API base URL.

Acceptance Criteria

- ☐ The live deployed frontend loads correctly and successfully calls the live deployed backend.
- ☐ A full manual smoke test (signup through result display) succeeds on the live URL.
- ☐ No CORS errors appear in the browser console when using the live app.

Edge Cases & Notes

- Test the deployed app on an actual mobile device over cellular data, not only desktop Wi-Fi.

PROMPT 202 — Deploy the React frontend to Vercel (or equivalent) and verify the build succeeds

Objective: Deploy the React frontend to Vercel (or equivalent) and verify the build succeeds.

Why This Matters: This step makes the personalized app actually reachable by real users at a real URL, pointed at the real deployed backend — completing the full path described in the system architecture chapter of the report.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Deploy the React frontend to Vercel (or equivalent) and verify the build succeeds.
- Confirm the production build uses the deployed backend's URL, not `localhost`.
- Verify HTTPS is active end-to-end (both frontend and backend) before considering the deployment complete.
- Confirm environment-specific config is injected at build/deploy time, not hard-coded in source.
- Keep a rollback path available (previous deployment/build) in case the new deploy breaks something.

Technical Specification

Target platform: Vercel (or equivalent static/SSR host for a Vite React app). Required env var: production API base URL.

Acceptance Criteria

- ☐ The live deployed frontend loads correctly and successfully calls the live deployed backend.
- ☐ A full manual smoke test (signup through result display) succeeds on the live URL.
- ☐ No CORS errors appear in the browser console when using the live app.

Edge Cases & Notes

- Test the deployed app on an actual mobile device over cellular data, not only desktop Wi-Fi.

PROMPT 203 — Configure a custom domain (or confirm the platform-provided URL) and HTTPS

Objective: Configure a custom domain (or confirm the platform-provided URL) and HTTPS.

Why This Matters: This step makes the personalized app actually reachable by real users at a real URL, pointed at the real deployed backend — completing the full path described in the system architecture chapter of the report.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Configure a custom domain (or confirm the platform-provided URL) and HTTPS.
- Confirm the production build uses the deployed backend's URL, not `localhost`.
- Verify HTTPS is active end-to-end (both frontend and backend) before considering the deployment complete.
- Confirm environment-specific config is injected at build/deploy time, not hard-coded in source.
- Keep a rollback path available (previous deployment/build) in case the new deploy breaks something.

Technical Specification

Target platform: Vercel (or equivalent static/SSR host for a Vite React app). Required env var: production API base URL.

Acceptance Criteria

- ☐ The live deployed frontend loads correctly and successfully calls the live deployed backend.
- ☐ A full manual smoke test (signup through result display) succeeds on the live URL.
- ☐ No CORS errors appear in the browser console when using the live app.

Edge Cases & Notes

- Test the deployed app on an actual mobile device over cellular data, not only desktop Wi-Fi.

PROMPT 204 — Verify CORS is correctly configured between the deployed frontend and backend origins

Objective: Verify CORS is correctly configured between the deployed frontend and backend origins.

Why This Matters: This step makes the personalized app actually reachable by real users at a real URL, pointed at the real deployed backend — completing the full path described in the system architecture chapter of the report.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Verify CORS is correctly configured between the deployed frontend and backend origins.
- Confirm the production build uses the deployed backend's URL, not `localhost`.
- Verify HTTPS is active end-to-end (both frontend and backend) before considering the deployment complete.
- Confirm environment-specific config is injected at build/deploy time, not hard-coded in source.
- Keep a rollback path available (previous deployment/build) in case the new deploy breaks something.

Technical Specification

Target platform: Vercel (or equivalent static/SSR host for a Vite React app). Required env var: production API base URL.

Acceptance Criteria

- ☐ The live deployed frontend loads correctly and successfully calls the live deployed backend.
- ☐ A full manual smoke test (signup through result display) succeeds on the live URL.
- ☐ No CORS errors appear in the browser console when using the live app.

Edge Cases & Notes

- Test the deployed app on an actual mobile device over cellular data, not only desktop Wi-Fi.

PROMPT 205 — Run a full smoke test of the live deployed app

Objective: Run a full smoke test of the live deployed app: signup, login, scan, dashboard, history, profile.

Why This Matters: This step makes the personalized app actually reachable by real users at a real URL, pointed at the real deployed backend — completing the full path described in the system architecture chapter of the report.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Run a full smoke test of the live deployed app: signup, login, scan, dashboard, history, profile.
- Confirm the production build uses the deployed backend's URL, not `localhost`.
- Verify HTTPS is active end-to-end (both frontend and backend) before considering the deployment complete.
- Confirm environment-specific config is injected at build/deploy time, not hard-coded in source.
- Keep a rollback path available (previous deployment/build) in case the new deploy breaks something.

Technical Specification

Target platform: Vercel (or equivalent static/SSR host for a Vite React app). Required env var: production API base URL.

Acceptance Criteria

- ☐ The live deployed frontend loads correctly and successfully calls the live deployed backend.
- ☐ A full manual smoke test (signup through result display) succeeds on the live URL.
- ☐ No CORS errors appear in the browser console when using the live app.

Edge Cases & Notes

- Test the deployed app on an actual mobile device over cellular data, not only desktop Wi-Fi.

4.G Documentation

PROMPT 206 — Write the root README.md with project overview, architecture diagram, and links to backend...

Objective: Write the root `README.md` with project overview, architecture diagram, and links to backend/frontend/ml docs.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `README.md`

Detailed Requirements

- Write the root `README.md` with project overview, architecture diagram, and links to backend/frontend/ml docs.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 207 — Write docs/SETUP.md with full local-development setup instructions for all three subprojects

Objective: Write `docs/SETUP.md` with full local-development setup instructions for all three subprojects.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `docs/SETUP.md`

Detailed Requirements

- Write `docs/SETUP.md` with full local-development setup instructions for all three subprojects.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.

- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 208 — Write docs/ARCHITECTURE.md documenting the full system architecture, matching the diagrams...

Objective: Write `docs/ARCHITECTURE.md` documenting the full system architecture, matching the diagrams in the project report.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `docs/ARCHITECTURE.md`

Detailed Requirements

- Write `docs/ARCHITECTURE.md` documenting the full system architecture, matching the diagrams in the project report.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 209 — Write docs/API_REFERENCE.md documenting every backend endpoint with request/response examples

Objective: Write `docs/API_REFERENCE.md` documenting every backend endpoint with request/response examples.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `docs/API_REFERENCE.md`

Detailed Requirements

- Write `docs/API_REFERENCE.md` documenting every backend endpoint with request/response examples.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 210 — Write docs/DATABASE_SCHEMA.md documenting all four tables and their relationships

Objective: Write `docs/DATABASE_SCHEMA.md` documenting all four tables and their relationships.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `docs/DATABASE_SCHEMA.md`

Detailed Requirements

- Write `docs/DATABASE_SCHEMA.md` documenting all four tables and their relationships.

- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 211 — Compile the final project report chapters with final, measured accuracy/evaluation numbers f...

Objective: Compile the final project report chapters with final, measured accuracy/evaluation numbers filled in.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Compile the final project report chapters with final, measured accuracy/evaluation numbers filled in.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 212 — Prepare architecture and flow diagrams as clean, slide-ready images for the presentation

Objective: Prepare architecture and flow diagrams as clean, slide-ready images for the presentation.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Prepare architecture and flow diagrams as clean, slide-ready images for the presentation.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.
- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

PROMPT 213 — Write docs/DEMO_SCRIPT.md — a step-by-step script for the live demonstration during evalua...

Objective: Write `docs/DEMO_SCRIPT.md` — a step-by-step script for the live demonstration during evaluation.

Why This Matters: Documentation is what lets anyone — a teammate, an evaluator, or future-you — understand and operate the system without having to reverse-engineer it from code. It is also direct, citable evidence of engineering rigor in the final evaluation.

Primary File(s): `docs/DEMO_SCRIPT.md`

Detailed Requirements

- Write `docs/DEMO_SCRIPT.md` — a step-by-step script for the live demonstration during evaluation.
- Write for a reader who has not seen the code before — define acronyms and name exact file paths/commands.

- Keep the document consistent with the actual current state of the system (re-check after any late changes elsewhere in the project).
- Use concrete examples (real curl commands, real field names) rather than abstract descriptions.
- Cross-link to related documents (e.g. API Reference links to Database Schema) rather than duplicating content.

Technical Specification

Target file lives under `docs/` at the project root, matching the path named in the objective above.

Acceptance Criteria

- ☐ A teammate unfamiliar with this specific module can follow the document and successfully complete the described task.
- ☐ Every command/example given in the document has actually been run and confirmed to work.
- ☐ The document is linked from the root `README.md` or another relevant index.

Edge Cases & Notes

- Flag any part of the system this document describes that is still incomplete or known to differ from the written description.

4.H Presentation & Viva Preparation

PROMPT 214 — Build the presentation slide deck (title, problem, methodology, architecture, results, demo,...

Objective: Build the presentation slide deck (title, problem, methodology, architecture, results, demo, limitations, future scope).

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Build the presentation slide deck (title, problem, methodology, architecture, results, demo, limitations, future scope).
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 215 — Prepare a 2–3 minute scripted live-demo walkthrough covering signup through result display

Objective: Prepare a 2–3 minute scripted live-demo walkthrough covering signup through result display.

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Prepare a 2–3 minute scripted live-demo walkthrough covering signup through result display.
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.

- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 216 — Prepare answers to likely viva questions

Objective: Prepare answers to likely viva questions: why CNN, why transfer learning, why FastAPI, why PostgreSQL, what is overfitting, what is precision/recall/F1.

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Prepare answers to likely viva questions: why CNN, why transfer learning, why FastAPI, why PostgreSQL, what is overfitting, what is precision/recall/F1.
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 217 — Prepare answers explaining the authentication and personalization architecture (JWT, per-use...

Objective: Prepare answers explaining the authentication and personalization architecture (JWT, per-user data scoping).

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Prepare answers explaining the authentication and personalization architecture (JWT, per-user data scoping).
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 218 — Rehearse the demo end-to-end at least twice, including a fallback plan (e

Objective: Rehearse the demo end-to-end at least twice, including a fallback plan (e.g. screen recording) in case of live network issues.

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Rehearse the demo end-to-end at least twice, including a fallback plan (e.g. screen recording) in case of live network issues.
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.

- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 219 — Do a final walkthrough of the entire report for consistency between the written claims and t...

Objective: Do a final walkthrough of the entire report for consistency between the written claims and the actual measured results.

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Do a final walkthrough of the entire report for consistency between the written claims and the actual measured results.
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.

PROMPT 220 — Final team review

Objective: Final team review: confirm every member can explain every phase of the pipeline, not just their own module.

Why This Matters: This is how the whole ten-week effort is actually communicated and defended in the viva — a technically excellent system that cannot be clearly explained or demonstrated under time pressure will still be under-credited.

Primary File(s): See the module file structure for this phase.

Detailed Requirements

- Final team review: confirm every member can explain every phase of the pipeline, not just their own module.
- Keep the content grounded in what was actually built and measured — never claim a number or feature that wasn't actually achieved.
- Rehearse against a timer if a time limit applies to the demo/presentation.
- Prepare for at least one thing going wrong live (network failure, slow model load) with a fallback plan.
- Make sure every team member, not just the one who built a given module, can speak to it.

Technical Specification

Draws on material already produced in `docs/`, the final project report, and the live deployed application for any demo component.

Acceptance Criteria

- ☐ The material described in the objective exists in a reviewable, shareable form (slide file, script document, etc.).
- ☐ At least one full dry run has been completed using this material.
- ☐ Any team member can present/answer questions on this material, not only its original author.

Edge Cases & Notes

- Have a non-live fallback (e.g. a short screen recording of the demo) ready in case the live environment fails during evaluation.
